



Aether

A Spotify companion that turns a listening session into responsive cosmic motion — designed, built, and profiled solo: hand-rolled OAuth, real-time Web Audio analysis, and a performance pass that took first load from **15.9 seconds to 3.4**.

[Live demo ↗](#) [Source ↗](#)

Next.js · Web Audio API · Spotify Web API · Vercel



PROBLEM

Listening is treated as a

utility, not an experience

Most streaming interfaces are the same regardless of what's playing: album art, a progress bar, a queue. The music never touches the screen.

Nothing in a typical player responds to the actual audio — a ballad and a breakbeat get identical chrome. The visual layer is decoration, not instrument.

SOLUTION

Aether turns audio into responsive

cosmic motion, light, and color

A canvas starfield and particle field that react to music in real time, wrapped around a working Spotify integration.

Beat detection drives particle color, motion, and full-screen light pulses as the track plays. Underneath that, Aether is a real Spotify client — playback control, search, saved tracks, playlists — reached through a boot sequence that draws its own decagonal mark into being before the dashboard ever appears, the same reveal technique used in miniature at the top of this page.

TECHNICAL CHALLENGES

Four constraints that shaped the build

I. Spotify's OAuth restrictions

The Authorization Code flow is hand-rolled — no auth library — with `httpOnly` cookie sessions, a signed CSRF `state` nonce, and automatic token refresh. That refresh logic was originally copy-pasted across three separate routes; consolidating it into one shared utility after the fact is the kind of thing a solo project lets slip until you go looking for it.

The platform itself sets the harder limit: Spotify caps unverified apps at five allow-listed accounts, and — under its 2026 policy — requires the app owner's own account to carry an active Premium subscription just to keep playback alive. The demo link above works for anyone; logging into it with your own Spotify account doesn't, by Spotify's design, not this app's.

II. Web Audio analysis

Spotify's stream is DRM/EME-protected — the Web Audio API cannot see inside it, on any app. Spotify's own Audio Features and Audio Analysis endpoints, which used to expose tempo and beat position directly, were deprecated for new apps in November 2024, closing the one workaround that used to exist.

Aether analyzes a local audio file instead, through a real `AnalyserNode` — frame-accurate FFT data, genuine beat detection off the bass band against a rolling baseline. The tradeoff is explicit and stated in the UI: the visualizer reacts to a track you hand it, not necessarily whatever is streaming from Spotify at that moment.

III. Animation performance

A particle system running continuously is an easy way to quietly own the main thread. The fix wasn't one change but a sequence of them — deferring the whole visual engine until it's actually visible, cutting per-frame React re-renders out of the custom cursors, short-circuiting the particle-distance math before its expensive step, and right-sizing images that were shipping ten times larger than they needed to be. The full arc, measured, is the next section.

IV. Accessibility and responsive design

A beat-reactive light show sits close to a real photosensitivity hazard, so the flash is engineered against the WCAG general flash threshold specifically: rate-limited under three triggers per second independent of how fast the music actually beats, hue clamped away from the red-flash danger zone, and skipped entirely under `prefers-reduced-motion` — not just dimmed.

Layout was audited at three real breakpoints — a modern phone, a tablet, and desktop — and came back clean, because the grid's column breakpoint and the Spotify panel's overflow handling were designed against each other from the start rather than patched in afterward.

MEASURED IMPACT

Three Lighthouse audits, same live deployment

Not simulated — three real runs against the production URL, each after a targeted round of fixes.



Lighthouse Performance score, deployed site, each round after a distinct fix pass.

Metric	Round 1	Round 2	Round 3
Accessibility	96	96	100
Largest Contentful Paint	15.9s	3.8s	3.4s
Time to Interactive	15.9s	3.8s	3.4s
Total Blocking Time	290ms	420ms	140ms
Speed Index	6.5s	2.8s	1.1s
Cumulative Layout Shift	0	0	0

Round 2's Total Blocking Time briefly regressed (290ms → 420ms) as a side effect of deferring more work past first paint — resolved in round 3 once the custom cursors stopped re-rendering through React on every mouse move.

KEY ENGINEERING DECISION

Nothing behind an opaque

overlay needs to be alive

The single largest cost the first audit surfaced wasn't a bug. It was a sequencing mistake.

FINDING

Lighthouse's own trace named the culprit precisely: one script chunk responsible for **2,322ms of pure scripting** — nearly four times the next-largest — traced back to the starfield and particle-field engine. It was running continuously from the moment the page loaded.

INSIGHT

That engine was running the entire time the intro's fully opaque boot sequence sat on top of it. Nothing about it was visible. The main thread was being spent animating a scene the user could not see and had not asked for yet.

MECHANISM

The intro's boot wrapper now reports, once, the exact moment it resolves to showing the real app — either immediately, if you've seen it before, or when the reveal animation finishes. The root layout gates canvas creation, the cursor engines, and the audio-reactive controller behind that single signal instead of mounting them unconditionally.

RESULT

The dominant script disappeared from the profile entirely. The largest remaining task after the change was **186ms** — Next.js's own bundler runtime, not application code — with zero visible change to the experience once someone actually enters.

2,322ms → 186ms

Largest main-thread script, before and after deferring initialization

Built solo, front to back — architecture, OAuth, audio analysis, and the performance work documented above.

[View the live site](#) [Read the source](#)